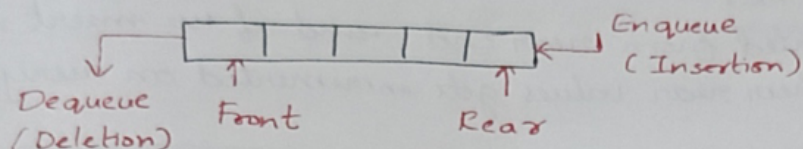


## QUEUES

- \* A queue can be defined as ordered list which enables insert operations to be performed at one end called REAR and delete operations to be performed at another end called FRONT
- \* Queue can be referred as to be first-in-first out list.



- \* A Queue Data Structure is a fundamental concept in computer science used for storing and managing data in a specific order.
- \* It follows the principle of "First in, First Out" (FIFO) where the first element added to the queue is the first one to be removed.

### Complexity of queue :-

| <u>Average access</u> | <u>Search</u> | <u>Insertion</u> | <u>Deletion</u> | <u>Space complexity</u> |
|-----------------------|---------------|------------------|-----------------|-------------------------|
| $O(n)$                | $O(n)$        | $O(1)$           | $O(1)$          | <u>Worst</u> $O(n)$     |
| <u>Worst access</u>   | <u>Search</u> | <u>Insertion</u> | <u>Deletion</u> |                         |
| $O(n)$                | $O(n)$        | $O(1)$           | $O(1)$          |                         |

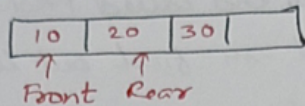
### Operations on Queue :-

- ① Enqueue :- Enqueue is used to insert element at rear end of queue. It returns void.
- ② Dequeue :- Dequeue operations performs the deletion from front end of queue. The dequeue operation can also be designed to void.
- ③ Peek :- This returns, element which is pointed by front pointer in the queue but does not delete it.
- ④ Queue Overflow (is full) :- When queue is completely full, then it shows overflow condition.
- ⑤ Queue Underflow (is empty) :- When there is no element in the queue then it throws underflow condition.



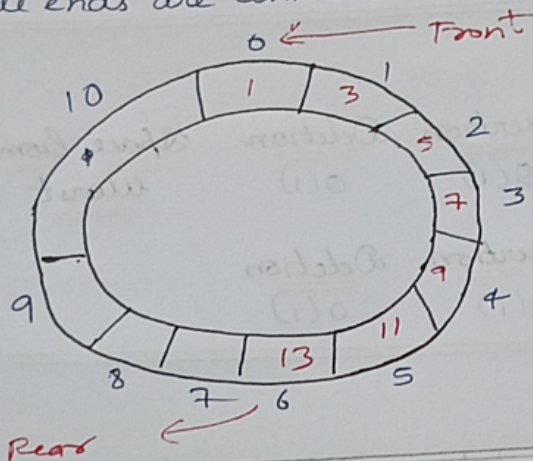
### Types of Queue :-

① Linear queue :- In linear queue, an insertion takes place from one end while deletion occurs from another end. It strictly follows FIFO rule. The linear queue can be represented, as



The elements are inserted from rear end, and if we insert more elements in queue, their rear values get incremented on every insertion.

② Circular queue :- In circular queue, all nodes are represented as circular. It is similar to linear queue except that last element of the queue is connected to the first element. It is also known as ring buffer as all ends are connected to another end.



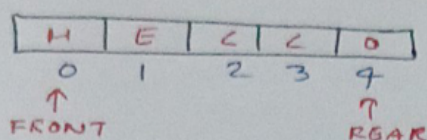
Draw back of Linear queue :- Insertion is done only from rear end. The Linear queue shows the overflow condition as rear is pointing to last element of the queue.

Draw back of Circular queue :- Drawback of linear queue is overcome in circular queue. If empty space is available, new element can be added i.e. empty space by simply incrementing value of rear.



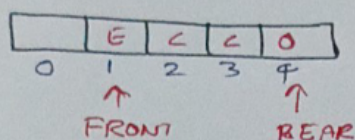
③ Priority queue :- The queue in which each element has some priority associated with it. Based on priority of the element, elements are arranged in a priority queue. If elements occur with some priority, then they are served according to FIFO principle.

Array representation of queue :-



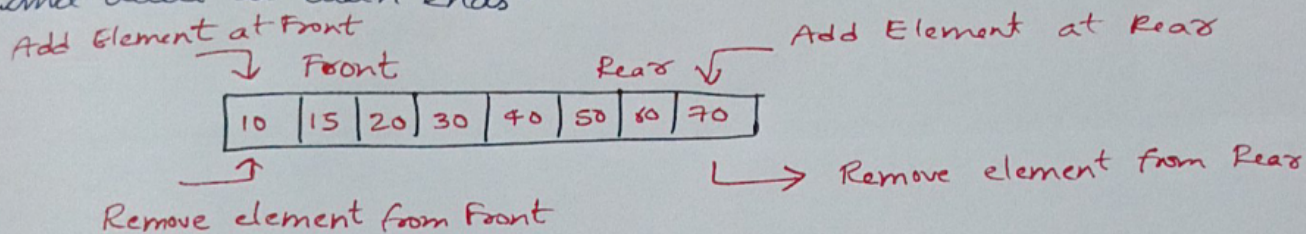
$\therefore$  Queue after inserting an element

After deleting element, value of front will increase from -1 to 0, the queue will look like



$\therefore$  Queue after deleting an element

④ Deque (or) Double Ended Queue :- Deque or Double Ended Queue is a generalized version of Queue data structure that allows insert and delete at both ends.

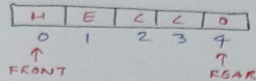


- \* Deque can act as both Stack and Queue.
- \* It is useful in many problems where we need to have a subset of all operations also like insert / remove at front and insert / remove at the end.
- \* It is typically implemented either using a doubly linked list or circular array.
- \* Deque Interface present in java.util package is a subtype of the queue interface.
- \* It can either be used as queue or as a stack.



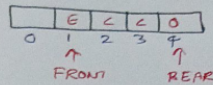
③ Priority queue :- The queue in which each element has some priority associated with it. Based on priority of the element, elements are arranged in a priority queue. If elements occur with some priority, then they are served according to FIFO principle.

Array representation of queue :-



∴ Queue after inserting an element

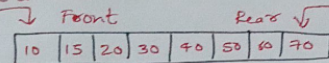
After deleting element, value of front will increase from -1 to 0, the queue will look like



∴ Queue after deleting an element

④ Deque (or) Double Ended Queue :- Deque or Double Ended Queue is a generalized version of Queue data structure that allows insert and delete at both ends.

Add Element at Front



Add Element at Rear

Remove element from Front

Remove element from Rear

- \* Deque can act as both Stack and Queue.
- \* It is useful in many problems where we need to have a subset of all operations also like insert / remove at front and insert / remove at the end.
- \* It is typically implemented either using a doubly linked list or circular array.
- \* Deque Interface present in java.util package is a subtype of the queue interface.
- \* It can either be used as queue or as a stack.



- Q1) K Queue
- Q2) K Reverse in a Queue \*
- Q3) Implement Stack using Queue \* \*
- Q4) Implement Queue using Stack \* \*
- Q5) 1st Non-Repeating in Stream
- Q6) Reverse 1st k elements of Queue \*
- Q7) Time Needed to Buy Tickets
- Q8) Rotten Oranges
- Q9) Sliding Window Maximum \*
- Q10) Evaluate Reverse Polish Notation
- Q11) Cycle Queue
- Q12) Deque \*
- Q13) Priority Queue (Heap) \*

